8. Introduction

Monitoring dashboards love to lie by omission. A hundred thousand servers can all show green while a handful of them have quietly stopped receiving any traffic — no crash, no error, no alert. The only way to catch that is to ask a very specific question, over and over, for different time windows: "which servers got *zero* requests between this time and that time?"

That's the real-world shape of LeetCode 2747, Count Zero Request Servers. It looks like a logs-and-timestamps exercise, but it's actually testing something interviewers care about a lot: can you recognize when a per-query brute force is going to blow up at scale, and can you restructure the problem so that sorting once lets you answer thousands of queries almost for free?

This problem is a great one to actually sit with, because the "obvious" data structure — a per-server last-seen timestamp — turns out to be a trap. The faster solution requires you to stop thinking per-server and start thinking in terms of counts and a sliding window. That shift in framing is the whole lesson here, and it shows up again and again in interview problems that look like "count things active in a range."

9. Problem Overview

You're given:

- `n` — the total number of servers, numbered `1` to `n`.
- `logs` — a list of `[server_id, time]` pairs, each one recording that a server received a request at a given time. These are not necessarily sorted.
- `x` — the length of the lookback window.
- `queries` — a list of timestamps. For each query time `q`, you need to count how many servers received **zero** requests during the window `[q - x, q]`, inclusive on both ends.

Return an array of answers, one per query, in the same order the queries were given.

Constraints matter here: `n`, the number of logs, and the number of queries can each be up to `10^5`. Any algorithm that re-examines the full log for every query is doing up to `10^10` operations in the worst case — far too slow. You're also guaranteed `x < queries[i]` for every query, which just means the left edge of the window never goes negative.

10. Example

Say `n = 3` and the sorted logs are:

server	time
1	3
1	5
2	6

With `x = 5`:

- **Query `q = 10`**: window is `[5, 10]`. The log at time 3 falls outside it. Logs at time 5 (server 1) and time 6 (server 2) are inside. So servers 1 and 2 are active, server 3 is silent → answer `1`.
- **Query `q = 11`**: window is `[6, 11]`. Now only the log at time 6 (server 2) is inside — the log at time 5 just aged out. Server 2 is active; servers 1 and 3 are silent → answer `2`.

Notice how the window *slides*: as `q` grows, old logs drop out on the left and new ones (if any) would enter on the right. That sliding behavior is the key to the efficient solution.

11. Intuition

The naive instinct is to treat this as a per-server problem, because the question is phrased per-server ("which servers are silent"). That pulls you toward a hash map of `server_id -> last_seen_time`, checked against each query. But to answer "who's silent" with that structure, you'd have to either scan all `n` servers per query, or maintain some kind of expiring structure for every server — and clearing stale entries for up to `10^5` servers on every one of up to `10^5` queries is exactly the `O(n * queries)` blow-up you're trying to avoid.

The better move is to stop asking "which servers" and start asking "how many are active," because the two are related by a trivial identity:

```
silent_count = n - active_count
```

If you can track `active_count` cheaply as the query window slides, you never need to know server identities at all for the final count — just the number.

To track `active_count` cheaply, sort the logs by time and sort the queries by time (while remembering each query's original index, since the answer array needs to come back in the original order). Once both are sorted, every query's window only ever moves forward relative to the previous query's window — it never needs to rewind. That's the property that makes a two-pointer sliding

window possible: a pointer that only increases, touching each log entry at most twice total (once to add it, once to remove it), across *all* queries combined.

Keep a `count[server]` array. When a server's count goes from `0` to `1`, it just became active — increment `active_count`. When a log exits the window on the left and a server's count drops from `1` to `0`, it just went silent — decrement `active_count`. That's the entire mechanism.

12. Brute Force Solution

Idea: For every query, scan the entire log, mark which servers appear within the window, then count how many servers never got marked.

Algorithm: 1. For each query `q`: - Initialize a `seen` set. - Scan every log entry; if its time is within `[q - x, q]`, add its server id to `seen`. - Answer is `n - len(seen)`.

Advantages: Dead simple to write and reason about; no sorting required; low risk of off-by-one bugs.

Disadvantages: Re-does the same scan for every query, recomputing overlapping work constantly.

Complexity: `O(queries * logs)` time, `O(n)` space per query. With both at `10^5`, that's up to `10^10` operations — far too slow for the given constraints.

13. Optimal Solution

The optimal approach has three moving pieces: a sorted log array, a sorted query list (index-preserved), and two pointers that only move forward.

Step 1 — Sort. Sort `logs` by timestamp. Sort query indices by the query's value, so you process queries from earliest window to latest.

Step 2 — Set up state. Create `count = [0] * (n + 1)` (one slot per server) and `active = 0`. Create two pointers, `left = 0` and `right = 0`, both indexing into the sorted logs.

Step 3 — Slide for each query, in time order:

Action	Condition	Effect
Advance `right`	`logs[right].time <= q`	Entering the window: increment `count[server]`; if it went `0 → 1`, `active += 1`
Advance `left`	`logs[left].time < q - x`	Leaving the window: decrement `count[server]`; if it went `1 → 0`, `active -= 1`

Step 4 — Record the answer. `answer[original_index] = n - active`.

Because `right` only ever moves forward and `left` only ever moves forward, each log entry is touched by `right` exactly once (entering) and by `left` exactly once (leaving), across the *entire* run of queries — not per query. That's what collapses the total work from `O(queries * logs)` down to `O(logs + queries)` after the sorts.

```
time ----3-----5---6----------------10----11---->
logs:     [1]  [1] [2]
queries:                           q=10  q=11
window for q=10:        [5 ................10]
window for q=11:            [6 ..................11]
```

14. Python Code

```
from typing import List


def count_zero_request_servers(
    n: int,
    logs: List[List[int]],
    x: int,
    queries: List[int],
) -> List[int]:
    logs_sorted = sorted(logs, key=lambda log: log[1])
    query_order = sorted(range(len(queries)), key=lambda i: queries[i])

    answer = [0] * len(queries)
    count = [0] * (n + 1)
    active = 0
    left = right = 0
    total_logs = len(logs_sorted)

    for i in query_order:
        q = queries[i]

        # Bring in logs that just entered the window.
        while right < total_logs and logs_sorted[right][1] <= q:
            server = logs_sorted[right][0]
            if count[server] == 0:
                active += 1
            count[server] += 1
            right += 1

        # Drop logs that have aged out of the window.
        while left < total_logs and logs_sorted[left][1] < q - x:
            server = logs_sorted[left][0]
            count[server] -= 1
            if count[server] == 0:
                active -= 1
```

```
            left += 1

        answer[i] = n - active

    return answer
```

15. Code Walkthrough

- `logs_sorted` — logs ordered by time so the window can only move forward.
- `query_order` — indices of `queries`, sorted by value, so queries are processed in time order while we still know where each answer belongs.
- `count` — one slot per server (1-indexed, hence `n + 1`), tracking how many logs for that server currently sit inside the window.
- `active` — running total of servers with `count[server] > 0`.
- First `while` loop — pulls `right` forward through every log entry that now falls at or before `q`; a transition from `0` to `1` in `count[server]` means that server just became active.
- Second `while` loop — pulls `left` forward through every log entry whose time is strictly less than `q - x`, meaning it has aged out; a transition to `0` means that server just went silent.
- `answer[i] = n - active` — write the result back at the query's *original* index, since we're iterating in sorted order.

16. Dry Run

Using `n = 3`, `logs = [[1,3],[1,5],[2,6]]`, `x = 5`, `queries = [10, 11]`.

- `logs_sorted` is already `[[1,3],[1,5],[2,6]]`. `query_order = [0, 1]` (10 before 11).
- **q = 10**, threshold for left = `10 - 5 = 5`.
- `right` loop: time 3 ≤ 10 → server 1, `count[1]: 0→1`, `active=1`. time 5 ≤ 10 → server 1, `count[1]: 1→2`, active stays. time 6 ≤ 10 → server 2, `count[2]: 0→1`, `active=2`. `right=3`.
- `left` loop: time 3 < 5 → server 1, `count[1]: 2→1`, active stays. time 5 is not `< 5` → stop. `left=1`.
- `answer[0] = 3 - 2 = 1`. ✓ matches expected.
- **q = 11**, threshold for left = `11 - 5 = 6`.
- `right` loop: nothing left to add, `right` stays at 3 (end of logs).
- `left` loop: time 5 < 6 → server 1, `count[1]: 1→0`, `active=1`. time 6 is not `< 6` → stop. `left=2`.
- `answer[1] = 3 - 1 = 2`. ✓ matches expected.

Final answer: `[1, 2]`.

17. Complexity Analysis

- **Time:** `O((logs + queries) log(logs + queries))` for the two sorts, plus `O(logs + queries)` for the sliding window itself, since `left` and `right` each advance at most `len(logs)` times total across the whole run, and we do `O(1)` work per query otherwise. Overall: `O((n_logs + n_queries) log(n_logs + n_queries))`.
- **Space:** `O(n + logs + queries)` for the `count` array, the sorted copies, and the answer array.
- **Why it's correct:** the two-pointer invariant holds because both logs and queries are sorted by time — the window's boundaries are monotonic non-decreasing functions of the query index in sorted order, so no pointer ever needs to move backward, and every log entry is added and removed exactly once over the life of the algorithm.

18. Alternative Solutions

A **Binary Indexed Tree (Fenwick tree) / prefix-sum-over-time** approach can also work: bucket logs by timestamp, build a structure that answers "how many distinct servers had any log in `[a, b]`" via range queries. It's viable and sometimes used when queries aren't allowed to be reordered (e.g., online queries arriving one at a time without the ability to batch-sort them first). But it adds real implementation complexity — tracking distinct servers in a range needs more than a simple sum — for no asymptotic win over the two-pointer approach in this offline-query setting. Stick with sort + sliding window unless you specifically need online queries.

19. Edge Cases

- **Empty logs:** every server is silent for every query; `active` never increments, so `n - active = n` throughout. Handled automatically.
- **x larger than the full time range:** the window just always contains everything seen so far; no special casing needed, pointers behave correctly.
- **Duplicate timestamps:** multiple logs can share a timestamp; the `<=` and `<` comparisons handle ties correctly as long as you're consistent about which boundary is inclusive.
- **A single server receiving all requests:** `active` maxes out at `1`; answer is always `n - 1`.
- **Minimum values** (`n = 1`, one log, one query): still correct — `count` has 2 slots (`0` unused, `1` used), window logic unaffected by scale.

- **Maximum constraints** (`10^5` each): this is exactly the case the optimal solution is built for; verify no accidental `O(n)` per-query work crept back in (e.g., don't rebuild `count` per query).

20. Common Mistakes

1. **Forgetting to preserve original query indices** before sorting — writes answers back in the wrong order.
2. **Using `<` instead of `<=` for the right pointer**, or `<=` instead of `<` for the left pointer — the window's inclusive/exclusive boundaries are asymmetric and easy to swap by accident.
3. **Reaching for a per-server last-seen map** instead of a count array — it's intuitive but leads to `O(n)` cleanup work per query.
4. **Resetting `count` or `active` inside the query loop** — this throws away all the amortized savings and turns the sliding window back into per-query brute force.
5. **Letting `left` or `right` move backward** — if your sort or indexing logic has a bug that lets a pointer reset, you silently reintroduce quadratic behavior or incorrect counts.
6. **Off-by-one on `n` indexing** — servers are typically 1-indexed; sizing `count` as `[0] * n` instead of `[0] * (n + 1)` causes an out-of-bounds write.

21. Interview Questions

- Why does sorting both logs and queries by time make a single forward-moving window sufficient?
- What breaks if you sort logs by server id instead of by time?
- How would you handle this if queries had to be answered online, one at a time, without batching?
- Can you adapt this window technique to answer "which specific servers are silent" rather than just the count?
- What's the time complexity if `x` varies per query instead of being fixed?

22. Similar Problems

- **LeetCode 239 — Sliding Window Maximum**: same core technique (monotonic window over sorted data), different aggregate (max instead of count).
- **LeetCode 1438 — Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit**: two-pointer window where you track a running property instead of rescanning.
- **LeetCode 1888 — Minimum Number of Flips to Make the Binary String Alternating**: another "maintain a running count as a window slides" problem.

- **LeetCode 2763 — Sum of Imbalance Numbers of All Subarrays**: offline-query-style counting with sorted structure assistance.

23. Key Takeaways

The expensive-looking part of this problem — per-query rescanning — disappears the moment you sort both logs and queries by time and notice the window only moves forward. Tracking counts instead of server identities turns "who is silent" into a cheap subtraction from a running active total. The pattern generalizes: whenever a problem asks you to answer many range-based queries against a fixed dataset, check whether sorting both sides lets you process everything with a single forward pass instead of restarting from scratch each time.

24. Watch the Video

For the full walkthrough — including a live timing comparison between the brute force and optimal solutions — watch the video here: {LINK}

25. About the Series

This breakdown is part of **Fun with Learning Technology**, a series that works through LeetCode's daily challenge and real engineering concepts side by side, favoring intuition over memorized patterns. Each episode traces a problem from first read to optimal solution, the same way you'd actually think through it at a whiteboard.

26. Call To Action

If this helped the problem click, give the video a like on YouTube and subscribe for more daily walkthroughs — and if you want these breakdowns delivered straight to your inbox, subscribe to the Substack. Drop a comment with your own approach or a problem you'd like covered next, and share this with anyone prepping for interviews.

Quick Review

What's the pattern tell for 'servers with no requests in a time window'?

Sliding window over sorted event times — when you need counts of absence within a moving range, sort then two-pointer the window.

Time complexity of the two-pointer sliding window solution?

O((n+q) log n) — sorting logs/queries dominates; the pointer sweep itself is linear.

Space complexity of the sliding-window approach?

O(n+q) for sorted copies, index arrays, and the frequency/count map of servers per window.

Trickiest bug when counting 'zero-request' servers per query?

Forgetting servers that never appear in logs at all — total servers minus active ones, not just window-absent ones.

Why does rescanning all logs per query fail here?

Constraints allow up to 1e5 queries and logs, so O(q*n) rescanning blows past limits — must reuse sorted order across queries.

Two pointers vs full resort per query — what's the key insight?

Queries sorted by time let left/right pointers only move forward, reusing prior work instead of recomputing the window from scratch.
